분류·결정트리·앙상블·랜덤포레스트

1장. 분류(Classification)의 개요

1) 분류 문제란?

- 입력 특성(Feature) X를 보고, 이산적인 클래스 레이블 y를 예측하는 지도학습
- 예: 스팸/정상 메일, 이탈/잔류, 불량/정상, 질병 유/무
- 다중 클래스: 붓꽃(3종), 이미지(10클래스) 등
- 다중 라벨: 한 샘플이 여러 클래스를 동시에 가짐(예: 사진에 사람+자동차)

2) 분류 모델의 출력 형태

- 결정(예측 클래스): 0/1, A/B/C ...
- 확률(또는 점수): $P(y=1|x)$ 같은 형태
  - 임계값(threshold)을 바꾸면 정밀도/재현율이 바뀜
  - 실무에서는 "확률"이 중요(리스크 기반 의사결정, 알림/차단 정책 등)

3) 데이터 관점에서 중요한 개념

- 클래스 불균형(imbalanced class)
  - 예: 정상 99%, 이상 1%
  - 정확도(accuracy)만 보면 "전부 정상"으로 예측해도 99%가 되어 착시 발생
  - 대안: F1, PR-AUC, 재현율 중심 평가, 리샘플링(SMOTE 등), 클래스 가중치 등
- 데이터 분할
  - Train / Validation / Test 분리
  - 교차검증(CV)으로 성능 추정 안정화
- 데이터 누수(leakage)
  - 미래 정보가 학습에 들어가거나, 전처리를 전체 데이터에 대해 fit하는 등
  - 성능이 비정상적으로 높으면 누수 의심

4) 분류 성능 평가 지표(핵심)

- 혼동행렬(Confusion Matrix)

    - TP: 실제 양성/예측 양성
    - TN: 실제 음성/예측 음성
    - FP: 실제 음성인데 양성으로 예측(오탐)
    - FN: 실제 양성인데 음성으로 예측(미탐)
- Accuracy(정확도) = (TP+TN)/(전체)
- Precision(정밀도) = TP/(TP+FP)
    - "양성이라고 한 것 중 진짜 양성 비율" → 오탐 비용이 큰 경우 중요
- Recall(재현율) = TP/(TP+FN)
    - "실제 양성 중 잡아낸 비율" → 미탐 비용이 큰 경우 중요(질병, 사기탐지)
- F1 = 2 *(Precision*Recall)/(Precision+Recall)
    - 불균형에서 타협 지표로 자주 사용
- ROC-AUC
    - TPR(재현율) vs FPR(오탐률) 곡선 면적
    - 불균형이 심하면 PR-AUC가 더 의미 있는 경우 많음
- 임계값 조정
    - 같은 모델이라도 threshold를 어디에 두느냐에 따라 운영 성능이 크게 바뀜

5) 대표 분류 알고리즘 큰 그림

- 선형 모델: 로지스틱 회귀(해석 용이, 빠름)
- 거리 기반: KNN(단순하지만 대규모/고차원에서 부담)
- 트리 기반: 결정트리(해석 용이), 랜덤포레스트/GBM(성능 강력)
- 커널 기반: SVM(데이터 규모 커지면 부담)
- 신경망: 데이터가 크거나 비정형(이미지/텍스트)에 강점

2장. 결정 트리(Decision Tree)

1) 부류 결정 트리 모델의 특징

- 규칙 기반 if-then 분기 구조로 분류/회귀 모두 가능
- 장점

    - 해석이 직관적(“이 조건이면 이 클래스”)
    - 비선형 관계를 잘 표현
    - 스케일링(표준화) 없이도 잘 동작하는 편
    - 특성 간 상호작용을 자연스럽게 모델링
- 단점
    - 과적합이 매우 잘 일어남(깊이 증가하면 학습데이터를 거의 외움)
    - 작은 데이터 변화에도 트리 구조가 크게 변할 수 있음(불안정)

2) 트리는 어떻게 분기(학습)하나? (핵심 직관)

- 목표: 노드(데이터 묶음)를 둘로 나눴을 때 “클래스가 더 잘 섞이지 않게(순수하게)” 만들기
- 분기 기준(불순도/순도 지표)
    - 지니 불순도(Gini impurity): $1 - \Sigma p_k^2$
    - 엔트로피(Entropy): $- \Sigma p_k \log p_k$
    - 정보이득(Information Gain): 분기 전후 불순도 감소량
- 가능한 분기 후보(특성, 임계값)를 훑어보며 불순도 감소가 큰 분기를 선택

3) 결정 트리 주요 파라미터(과적합 제어 장치)

- max_depth: 트리 최대 깊이 제한(깊으면 복잡도↑, 과적합↑)
- min_samples_split: 노드를 분기시키기 위한 최소 샘플 수
- min_samples_leaf: 리프(말단 노드)에 남겨야 하는 최소 샘플 수
    - 과적합 줄이는 데 매우 효과적
- max_features: 분기 시 고려할 특성 수 제한(랜덤성 부여, 일반화 도움)
- criterion: gini / entropy 등
- 가지치기(Pruning)
    - 사전 가지치기: 위 파라미터로 미리 제한
    - 사후 가지치기: 일단 키운 뒤 불필요한 가지 제거(예: cost-complexity pruning)

4) 결정 트리 모델의 시각화/해석 포인트

- 노드에 보통 표시되는 정보

    - 분기 조건(예: feature_i <= 2.45)
    - 불순도(gini/entropy)
    - 샘플 수(samples)
    - 클래스 분포(value)
    - 예측 클래스(class)
- 해석 팁
    - 상위 분기일수록 “중요하게 쓰는” 특성일 가능성이 큼
    - 다만 feature importance는 편향이 있을 수 있음

5) 결정 트리 과적합(Overfitting)

- 원인: 리프를 계속 쪼개면 학습 데이터의 잡음까지 설명 가능
- 증상: train 성능은 매우 높고, test 성능이 낮음
- 대응
    - max_depth 낮추기
    - min_samples_leaf / min_samples_split 키우기
    - 가지치기 적용
    - 교차검증으로 최적 파라미터 탐색
    - 단독 트리보다 앙상블(랜덤포레스트/GBM) 고려

6) (실습 맥락) 사용자 행동 인식 데이터 세트 포인트

- 센서/행동 인식 데이터는 특성이 많고 클래스가 여러 개일 수 있음
- 트리 기반은 비선형 패턴을 잘 잡지만 과적합도 빨라서
    - 기본 트리 성능 확인 → 제약/튜닝으로 일반화 성능 개선 → 앙상블로 확장

3장. 앙상블 학습(Ensemble Learning) + 보팅

1) 앙상블 학습 개요

- “약한 모델 여러 개”를 결합해서 “강한 모델”을 만드는 접근
- 목표
    - 편향(bias) 감소 또는 분산(variance) 감소

    - 일반화 성능 향상
- 대표 방식
    - 보팅(Voting): 서로 다른 모델의 예측을 투표로 결합
    - 배깅(Bagging): 데이터 샘플링으로 여러 모델 학습(대표: 랜덤포레스트)
    - 부스팅(Boosting): 이전 모델이 틀린 것에 집중하며 순차 학습(GBM, XGBoost 등)
    - 스태킹(Stacking): 여러 모델의 출력을 다시 메타 모델이 학습

2) 보팅 유형: 하드 보팅 vs 소프트 보팅

- 하드 보팅(Hard Voting)
    - 각 모델의 "최종 클래스"를 다수결로 결정
    - 장점: 단순, 확률 출력 없어도 가능
    - 단점: 확률 정보(확신 정도)를 버림
- 소프트 보팅(Soft Voting)
    - 각 모델의 클래스 확률을 평균(또는 가중 평균)해 최종 결정
    - 보통 하드 보팅보다 성능이 더 잘 나오는 경우가 많음
    - 전제: 확률 출력이 가능하거나 확률 보정(calibration)이 잘 되어야 함

3) 보팅 분류기(Voting Classifier) 구성 팁

- 서로 "다른 성향"의 모델을 섞을수록 이득이 큰 편
    - 예: 로지스틱 회귀(선형) + SVM + 결정트리/랜덤포레스트
- 개별 모델이 너무 비슷하면 앙상블 이득이 작을 수 있음
- 가중치(Weighted voting)
    - 성능 좋은 모델에 더 큰 가중치를 주어 결합 가능
- 주의
    - 데이터 누수 없이 교차검증으로 개별 성능 검증 후 결합
    - 소프트 보팅은 확률 보정이 중요할 수 있음

4장. 랜덤 포레스트(Random Forest)

1) 랜덤 포레스트의 개요

- 배깅(Bagging) 기반의 대표적인 트리 앙상블
- 핵심 아이디어 2개

  1) 부트스트랩 샘플링(bootstrap): 중복 허용 랜덤 샘플로 여러 트리 학습

  2) 특성 랜덤 선택(max_features): 각 분기에서 일부 특성만 보고 분기
- 효과
  - 결정트리의 약점(분산/불안정성)을 줄여 과적합을 완화
  - 단일 트리보다 일반화 성능이 대체로 좋음

2) 랜덤 포레스트 예측 방식

- 분류: 여러 트리의 예측을 다수결(또는 확률 평균)로 결합
- 회귀: 예측값 평균

3) 랜덤 포레스트 주요 하이퍼파라미터 및 튜닝 포인트

- n_estimators: 트리 개수(많을수록 안정화되나 연산량 증가)
- max_depth: 트리 깊이(깊으면 과적합 경향)
- max_features: 분기마다 고려할 특성 수(작게 하면 다양성↑)
- min_samples_split / min_samples_leaf: 과적합 완화에 도움
- class_weight: 불균형 데이터에서 중요(예: balanced)
- oob_score(out-of-bag)
  - 부트스트랩에서 뽑히지 않은 샘플(OOB)로 성능 추정

4) 랜덤 포레스트의 장단점

- 장점
  - 튜닝을 크게 안 해도 강력한 기본 성능
  - 스케일링에 비교적 둔감
  - 특성 중요도(importance) 제공
- 단점
  - 단일 트리보다 해석이 어렵고 모델이 무거움
  - 부스팅 계열(GBM 등)보다 성능이 밀릴 때도 있음